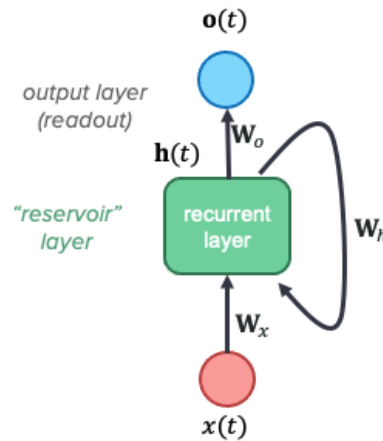


Additional Material (Lab3-2):

Echo State Networks



- Input $x(t)$: column vector of size N_x
- Reservoir state $h(t)$: column vector of size N_h
- Output $o(t)$: column vector of size N_o

When both input and output are 1-dimensional (e.g., as in the assignments), we use $N_x = N_y = 1$.

Reservoir

- It is a recurrent, non-linear layer with (typically) sparse connectivity among units.
- It computes the state transition function: how the state at time step t is obtained from the input at time step t and from the state at time step $t-1$:

$$h(t) = \tanh(W_h h(t-1) + W_x x(t) + b_h)$$

- Note that the bias vector can be included in the input-weight matrix W_x for simplicity, in this case, the input at each time-step should be augmented by a fixed input bias term equal to 1. In this case, the reservoir equation reads as follows:

$$h(t) = \tanh(W_h h(t-1) + W_x [x(t); 1]),$$

where $[;]$ denotes vector concatenation

- o Note that in this case the input bias for the reservoir is implemented by concatenating the input at each time step with a constant input bias equal to 1;
- Alternatively, the bias vector can be initialized as a random vector of the size of the reservoir, scaling its entries to have a maximum magnitude of a bias scaling hyper-parameter.
- Use a null state as initial state of the ESN: $h(0) = 0$

Readout

It computes the output function: how the output of the network at time step t is obtained from the state at time step t . In the simplest form, it is implemented as a feed-forward, linear dense layer, as follows:

$$o(t) = W_y[o(t); 1]$$

where the readout bias vector is already included in the output weight matrix W_y .

- Note that instead of using a readout explicitly implemented as in the above formula, you can use a readout in the form of any regressor / classifier (depending on the problem at hand). For instance, you can use Scikit-learn [RidgeClassifier](#) (for classification) or [Ridge](#) (for regression).

Reservoir Initialization

Initialize the reservoir according to the necessary condition for the Echo State Property (ESP), i.e., $\rho(W_h) < 1$. Recall that $\rho(\cdot)$ denotes the *spectral radius* (i.e., the maximum length of an eigenvalue) of its argument, namely $\rho(W_h) = \max(\text{abs}(\text{eig}(W_h)))$.

To properly initialize the reservoir weights, proceed as follows:

1. Initialization of the input weight matrix:
Choose the values of W_x from a random distribution scaled by a hyper-parameter ω_x . A typical choice is a uniform distribution on $(-\omega_x, \omega_x)$.
2. Initialization of the bias vector:
Choose the values of b from a random distribution scaled by a hyper-parameter ω_b . A typical choice is a uniform distribution on $(-\omega_b, \omega_b)$.
3. Initialization of the recurrent weight matrix:

- a. Initialize randomly the values of W_h from a random distribution (e.g., from a uniform distribution in $(-1, 1)$).
- b. Re-scale W_h to a unitary spectral radius, i.e., $W_h \leftarrow \frac{W_h}{\rho(W_h)}$
- c. Finally, re-scale W_h to the desired spectral radius ρ , i.e., $W_h \leftarrow \rho W_h$

Readout Training

Only the readout needs to be trained.

1. Discard an initial transient (run the network for some steps before starting to collect the states during training)
2. Collect all reservoir states and target values for each time step into matrices (after the initial transient)

$$H = [h(1), \dots, h(T)] \quad Y = [y(1), \dots, y(T)]$$

where $y(t)$ here denotes the target output at time-step t .

3. After having collected all the states, train the linear readout
 - Pseudo-inverse

$$W_o = Y H^+$$

- Ridge regression (λ_r is a regularization coefficient)

$$W_o = Y H^T (H H^T + \lambda_r I)^{-1}$$

note: the readout should not be trained after each time step, but only once after the collection of all the states

- Notice how in this case the reservoir states can be computed just once, as the recurrent layer's weights are not modified.

Implementation notes

- In NumPy, you can use the linear algebra functions in the numpy.linalg library (<https://numpy.org/doc/stable/reference/routines.linalg.html>) for computing the spectral radius and for training the readout (e.g., for inverse, pseudo inverse, etc)
- In Matlab, you can use the functions eig(.), pinv(.), and inv(.)

Reservoir Guesses

For every reservoir hyper-parametrization the performance (e.g. accuracy for classification task, MSE for regression task) should be averaged over a number of reservoir guesses (different random instantiations of networks with the same values of the hyper-parameters).

Model Selection!

To properly setup the network you need to use a proper model selection (e.g, by grid or random search), considering suitable values of the hyper-parameters. Relevant hyper-parameters might include the following: the number of reservoir units, spectral radius, input scaling, bias scaling, and readout regularization.